

Contents

1	Introduction	2
2	Package Architecture	2
2.1	Information Flow	2
2.2	Project Scaffolding	3
3	Example: CO₂ Injection System	4
3.1	Problem Set Up	4
3.2	Simulating with hilsim	5
3.2.1	Sensor Configuration File	5
3.2.2	Fake Sensors and Sensor Drivers	7
3.2.3	Actuator Configuration File	9
3.2.4	Actuator Drivers	10
3.2.5	Device Configuration File	12
3.2.6	World State Configuration File	12
3.2.7	Main Function	12
3.2.8	Results	14

1 Introduction

In this example, we will build a simple fluid injection system on top of the python library `hilsim`. This purpose of this tutorial is to 1) explain the general architecture of `hilsim` and 2) showcase how `hilsim` can be used to simulate a real system.

2 Package Architecture

2.1 Information Flow

`hilsim` is a python library designed for rapid hardware-in-the-loop (HITL) and software-in-the-loop (SITL) simulation. Each piece of hardware is described in a configuration file, followed by a model of the component in code. Finally the components interact with the system state from the main logic.

The general architecture of the `hilsim` is based on a common design pattern of a host-peripheral microcontroller relationship. A serial communication protocol is standardized between the host and all peripheral microcontrollers.

`hilsim` can accommodate an arbitrary number of peripheral microcontrollers, sensors, and actuators. `hilsim` takes the view that anything that changes the system state, whether intended or unintended is an actuator in the system. Both a solenoid designed to change fluid levels and a building's HVAC system causing inadvertent temperature fluctuations are viewed as actuators. Each can be modeled with relative ease in `hilsim`.

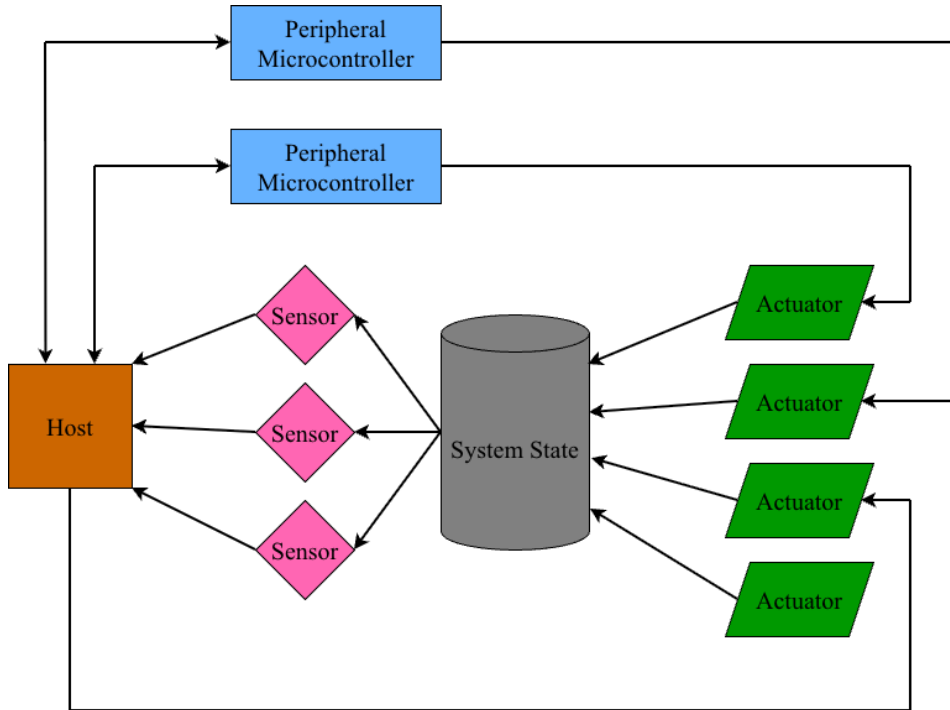


Figure 1: Information Flow in `hilsim`

Figure 1 describes the information flow in `hilsim`. Notice the two-way communication between the host and peripheral microcontrollers. Every time information is sent to a peripheral microcontroller from the host, the host expects the same information to be sent back as confirmation. In the above diagram, feedback loops are closed with the host. This is often favorable in low-speed systems, especially where prolific recording of sensor output is desired. However, it is possible to close the loop on peripheral microcontrollers without host/supervisor involvement, and often necessary for high-speed systems. It is possible but difficult to build this system architecture with the current version of `hilsim` (v0.1.3). Better support for intradevice communication in high speed systems (i.e. where it is necessary to close this loop on the microcontroller) will come in a future release.

2.2 Project Scaffolding

`hilsim` uses a predefined framework. Once the package has been installed with `pip install hilsim`, a number of commands become available to the user. `hilsim create <project_name>` (make sure to replace `<project_name>` with the name of your project) will generate the project scaffolding, including templates for sensors and actuators. Suppose we run `hilsim create my_project`. The initial scaffolding will appear as:

```
my_project
├── actuator_config.toml
├── actuators
│   └── actuator_template.py
├── device_config.toml
├── main.py
├── sensor_config.toml
├── sensors
│   └── sensor_template.py
└── world_state.toml
```

There are four major components to the scaffolding:

1. Four `.toml` file templates in the project root.
2. `main.py` in project root.
3. Two folders, called `sensors` and `actuators` in the project root, both of which contain templates for their respective devices.

Each component of the project scaffolding serves the following purpose:

- **actuator_config.toml:** This toml file describes each actuator that affects the state of the system. A detailed description of each field is provided in the generated template. The main purpose of this file is to 1) map the actuator to the device that controls it (i.e. a peripheral microcontroller) 2) describe the failure rates of the actuators (e.g. the probability of a solenoid getting stuck in its current state) 3) The latency between sending a command to the actuator and an actual change in the state of the actuator. 4) List the system variables affected by the actuator and denote the effect as persistent or transient, where persistent indicates the effect on the system variable lingers after the actuator changes state (e.g. the water level after a solenoid adds water to a reservoir) and transient indicates a change that is almost completely state dependent (e.g. the current drawn by the solenoid when open vs. closed.)
- **actuators/actuator_template.py:** This template is used to describe the actual physics of the actuator. The template class inherits from `ActuatorBase` which contains many of the underlying methods to interface actuators with the rest of the system. In the template the developer must write code to build commands that are sent to the peripheral microcontroller based on the target state. If the actuator is not controlled, the developer can simply `pass` the method or return `None`. The developer must also build an arbitrary number of methods that describe how the state of an actuator translates to the effect on the system state. Each method must follow the naming convention: `effect_<variable_name>` where `variable_name` is the name of the system state variable affected by that method.
- **device_config.toml:** This template describes the peripheral microcontrollers (or anything connected over a serial interface.) Under the hood it uses the `pyserial` to communicate. From the host's perspective, each device is really just a serial connection, hence why it is not necessary to write an associated class or driver for the device. When a device is simulated, there are two possible ways to simulate it: `sim`, or `super sim`. `Sim` uses the `socket` library to open virtual serial ports to communicate with the microcontroller code. This allows you to test both the host system and the actual microcontroller code. The `sim` mode is currently in development and has not been integrated on v0.1.3. The `super sim` mode uses a built-in fake serial module in python to "simulate the device" and return confirmations, simulate failures, etc. and does not require the actual microcontroller code. `Super sim` is currently supported on v0.1.3.

- **main.py**: This template hosts the the main logic of the program. It has access to a sensor manager, sensor tracker, the world state, the actuators (which are tied to devices), file managers, and rich output objects for viewing data in the terminal in real time.
 - **sensor.config.toml**: This template describes the sensors. Each field describes the constants of the sensor: how many variables it measures, the min, max, name, and unit of each variable it reads. It also controls the sim mode of each sensor and maps the sensor to its associated driver in the `sensors/` folder.
 - **sensors/sensors_template.py**: This template describes two things: the sensor driver and an instance of a “fake sensor.” When the sensors are initialized, the `device` attribute will either map to an instance of the actual sensor or the fake sensor (described below) at which point the driver becomes agnostic to the underlying device. Hence it is important that the fake sensor perfectly mirrors the methods of the real sensor class. The fake sensor, instead of reading actual physical values, pulls values from the simulated background dynamics.
 - **DriverTemplate**: The driver template standardizes sensors to three methods: `initialize_sim`, `initialize_real`, and `form_vals_dict`. The initialization methods set up the sensor, whereas the `form_vals_dict` method will either read the real sensor, or use the `FakeSensor` class to pull a value from the world state and return the results as dictionary where the key is the name of the variable and the value is the reading. The initialization methods are called at run time and do not need to be called by the user.
 - **FakeSensor**: The fake sensor template acts as a proxy to the simulated system dynamics when the sensor is set to sim mode. This class allows the driver to be agnostic to the device, whether it is reading actual values or pulling from the simulated world state.
 - **world_state.toml**: This template describes the world state and the initial values of the variables for simulation mode. It also includes the units of the variable. Note that these units should match those in the sensor configuration file, and any necessary conversions should be done in the driver and/or fake sensor.
-

3 Example: CO₂ Injection System

In sealed plant habitats, carbon dioxide (CO₂) is often depleted due to plant absorption. To maintain a healthy growing environment, it is necessary to inject (CO₂) into the system. The following system will demonstrate building a CO₂ injection system for a plant habitat.

3.1 Problem Set Up

Here we want to simulate plants consuming CO₂, reaching a critical threshold of 200 ppm, followed by an injection CO₂ until the concentration reaches a nominal level of 1200 ppm. Suppose it is known that opening the solenoid causes the the CO₂ in the system to increase by 500 ppm per second. We’ll further suppose that the solenoid is driven by a microcontroller using bare metal C to showcase the host-peripheral communication. To rapidly prototype this, we can use a Raspberry Pi 4b as the host, an RP2040 chip as the peripheral microcontroller, a SCD30 sensor to measure CO₂, an INA260 sensor to measure the system current, and a solenoid.

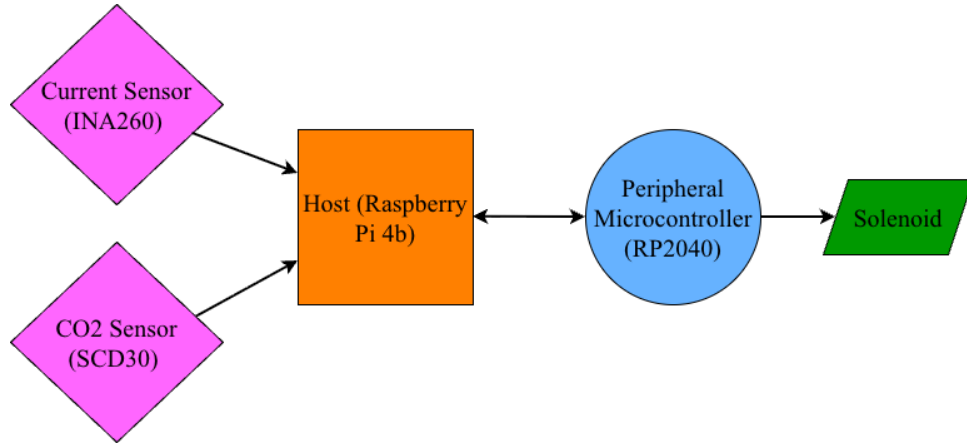


Figure 2: CO₂ Injection System

3.2 Simulating with hilsim

To build the simulation for this problem, we will need to:

1. Input the SCD30 and INA260 sensors into the sensor configuration file.
2. Write the drivers and fake sensor methods / classes for the SCD30 and INA260.
3. Input solenoid and plant actuators into the actuator configuration file.
4. Write the actuator methods / classes for the solenoid and plants.
5. Input the RP2040 into the device configuration file.
6. Fill out the world state configuration file.
7. Write the main logic.

At first, this seems like a lot. Fortunately, **hilsim** does the heavy lifting and most of this comes out to relatively simple work. We should be able to get a simulation going in a few minutes. We’ll walk through each of these steps.

3.2.1 Sensor Configuration File

In this example, we will be simulating all of the components. We will set the `sim_all` value to true so that this simulation can run on any computer and does not attempt to access an real i2c bus. For the sensor error program, we’ll assume that we only want to allow 4 subsequent transient errors before attempting some sort of recovery measure (either toggling SDA 9 times or power cycling) on every subsequent failed reading. Note that both of these methods are not implemented by default to keep the library hardware agnostic. Rather, the sensor tracker’s “track” method returns a dictionary of boolean values with keys “i2c_cycle” and “power_cycle” for which the user can use in the main loop. How to actually implement those recovery methods is not the focus of this example.

Suppose we have data that shows both sensors occasionally fail (about 0.2% of the time). We will reflect this in the `failure_rate` section. We will set the `sim_fail_initialization` to false. This allows fake sensors to simulate a failed initialization. If the `skip_failed_init` field is true, than a sensor that fails initialization is removed from the enabled sensors and effectively disappears. If `skip_failed_init` is false, then the program will raise an exception on a failed initialization.

We will fill out the readings section with all of the variables that the sensors reads (note that these sensors also read other variables, namely: temperature, relative humidity, voltage, and power.) This is necessary to keep the driver agnostic to the real/fake sensor. These variables become part of the system, and will need to be included in the world state. For noise and drift, we will assume there is a little bit in the SCD30 sensor, but none in the INA260 sensor. We end up with the following configuration file:

```

[sensor_params]
enabled_sensors = ["SCD30", "INA260"]
sim_all         = true
skip_failed_init = false

[sensor_error_program]
first_i2c_cycle   = 4
second_i2c_cycle  = 5
first_power_cycle = 6
second_power_cycle = 7
dead              = 8

[sensors.SCD30]
description          = "readings CO2"
num_measurements     = 1
driver_name          = "SCD30Driver"
fake_sensor_name     = "FakeSCD30"
sim                  = true
failure_rate         = 0.02
sim_fail_initialization = false

[[sensors.SCD30.readings]]
name      = "CO2"
units     = "ppm"
min       = 0
max       = 40000
noise     = 0.1
drift     = 0.1

[[sensors.SCD30.readings]]
name      = "relative_humidity"
units     = "percent"
min       = 0
max       = 95
noise     = 0.1
drift     = 0.1

[[sensors.SCD30.readings]]
name      = "temperature"
units     = "C"
min       = 0
max       = 50
accuracy  = 0.4
noise     = 0.1
drift     = 0.1

[sensors.INA260]
description          = "readings current"
num_measurements     = 1
driver_name          = "INA260Driver"
fake_sensor_name     = "FakeINA260"
sim                  = true
failure_rate         = 0.02
sim_fail_initialization = false

[[sensors.INA260.readings]]
name      = "voltage"
units     = "V"
min       = 0
max       = 40
noise     = 0.005
drift     = 0.1

```

```

[[sensors.INA260.readings]]
name      = "current"
units     = "mA"
min       = 0
max       = 15000
noise     = 0.1
drift     = 0.5

[[sensors.INA260.readings]]
name      = "power"
units     = "mW"
min       = 0
max       = 600000
noise     = 0.0005
drift     = 0.012

```

Note that the `driver_name` and `fake_sensor_name` are strings that match the name of driver and fake sensor classes in the `sensors/` folder.

3.2.2 Fake Sensors and Sensor Drivers

With the sensor information in the configuration file, we can proceed to write the code for these sensors. Both of these sensors can be picked up from Adafruit and have preexisting drivers. In this example we will wrap this code into our drivers. In the case of each sensor reading, we will include the possibility of failure (as described in the `failure_rate` from the configuration file.) To do this, we will call the `add_failure_possibility()` in any method of the `FakeSCD30` and `FakeINA260` classes that returns a a reading value. You'll notice that in both of these fake sensors, they perfectly mirror the methods of the preexisting drivers, allowing the driver to be agnostic to the device. Notice that the **name** of classes must exactly match those written in the sensor configuration file.

```

"""SCD30 Sensor"""

from datetime import datetime, timedelta
from random import random

from hilsim.core.sensors import DriverBase, SensorBase

# if sim=False, will import SCD30 from scd30_i2c

class FakeSCD30(SensorBase):
    """This is a class that has fake methods and produces fake data for the SCD30 sensor"""

    def set_measurement_interval(self, a: float) -> None:
        """Set how often the sensor measures"""
        self.measurement_interval = a
        # Could reflect 2 sec max rate here

    def start_periodic_measurement(self) -> None:
        """Begin periodic readings of the sensor"""
        self.reading = True
        self.last_reading_time = datetime.now()

    def get_data_ready(self) -> bool:
        """Determine if data is ready based on measurement_interval"""
        if datetime.now() - self.last_reading_time > timedelta(
            seconds=self.measurement_interval
        ):
            return True
        return False

    def read_measurement(self) -> list[float]:
        """Read a measurement (i.e. return values)"""

```

```

        # Note that if the sensor fails, all three readings fail to keep with
        # observed behavior

        self.add_failure_possibility()
        co2_val = self.get_return_value("CO2")
        rel_humid = self.get_return_value("relative_humidity")
        temp = self.get_return_value("temperature")

        return [co2_val, rel_humid, temp]

class SCD30Driver(DriverBase):
    """SCSD30 Driver that standardizes methods to initialize and read"""

    def initialize_sim(self) -> None:
        """Initialize sim mode for an SCD30 driver"""
        self.device = self.fake_sensor
        self.device.set_measurement_interval(0.1)
        self.device.start_periodic_measurement()

    def initialize_real(self) -> None:
        """Initialize real mode for an SCD30 Driver"""
        from scd30_i2c import SCD30 # type: ignore[import-not-found]
        self.device = SCD30()
        self.device.set_measurement_interval(0.1)
        self.device.start_periodic_measurement()

    def form_vals_dict(self) -> dict:
        """Create dictionary of sensor readings"""
        vals = self.device.read_measurement()
        return {"CO2": vals[0],
                "relative_humidity": vals[1],
                "temperature": vals[2]}
#- - - - -
"""The ina260 sensor"""

from hilsim.core.sensors import DriverBase, SensorBase
from hilsim.core.world_state import WorldState

class FakeINA260(SensorBase):
    """Fake INA260 that does not require physical hardware"""

    def __init__(self, config: dict, world_state: WorldState) -> None:
        super().__init__(config, world_state)
        self.voltage_val = 0.0
        self.current_val = 0.0

    @property
    def voltage(self) -> float | None:
        """Generate voltage value"""
        self.add_failure_possibility()
        self.voltage_val = self.get_return_value("voltage")
        return self.voltage_val

    @property
    def current(self) -> float:
        """Generate current value"""
        self.add_failure_possibility()
        self.current_val = self.get_return_value("current")
        return self.current_val

    @property
    def power(self) -> float | None:
        """Generate power value"""
        self.add_failure_possibility()

```



```

        return self.current_val self.voltage_val

class INA260Driver(DriverBase):
    """Driver for INA260 sensor, standardizes sensor to initialize and read"""

    def initialize_sim(self) -> None:
        self.device = self.fake_sensor

    def initialize_real(self) -> None:
        import adafruit_ina260 # type: ignore[import-not-found]
        self.device = adafruit_ina260.INA260(self.i2c_bus)

    def form_vals_dict(self) -> dict:
        voltage = self.device.voltage
        current = self.device.current
        power = self.device.power
        return {"voltage": voltage,
                "current": current,
                "power": power}

```

Note that I have left the `get_data_ready()` method out of the `read_measurement()` method for the FakeSCD30 sensor. I have done this in the interest of keeping this simulation as simple as possible while the actual sensor is limited to a maximum reading rate of 0.5Hz, which normally causes the sensor to throw an error when attempting readings faster than the maximum read rate. For simplicity we will assume that the sensor is always ready to read in full simulation mode. A higher fidelity example would incorporate the maximum reading rate.

In order to get the variable from the system dynamics simulation in a fake sensor class, call the `get_return_value()` method with the name of the variable as the argument. This argument is a string that must perfectly match the name of one of the sensor readings. The method `get_return_value()` also takes an optional secondary argument (that defaults to true) called `add_noise_and_drift`. If it is false, noise and drift is not included in the reading.

3.2.3 Actuator Configuration File

Here we have two actuators: plants and a solenoid. We have no control over the plants. We will assume that the linearly reduce the amount of CO₂ in the system by 5 ppm every second as long as they are alive. We won't account for their effect on any other system variable (primarily because we are not interested in that for this example.)

```

[Plants.settings]
name = "Plants"
driver = "Plants"
device = "None"
pin = "None"
sim = true
initial_state = "alive"
states = ["dead", "alive"]

[Plants.failure_sim]
stuck_probability = 0
cta_latency = {distribution="normal", mean=0, std=0}

[Plants.effects]
CO2 = {effect_types = "t+p+s"}

#- - - - -

[Solenoid.settings]
name = "Solenoid"
driver = "CO2Solenoid"
device = "RP2040"
pin = 8
sim = true
initial_state = "off"

```

```

states = ["on", "off"]

[Solenoid.failure_sim]
stuck_probability = 0.05
cta_latency      = {distribution="normal", mean=10, std=3}

[Solenoid.effects]
CO2      = {effect_types = "t+p+s"}
current  = {effect_types = "t+s"}

```

Since the plants are not controlled by any device, we set their device and pin to strings of “None”. However, they still have a driver, which is the class called “Plants” in the `actuators/` where their behavior is modeled. The plants cannot “get stuck” in their current state, so we can set the fields in `failure_sim` to anything we want. It will not ever be accessed because it requires a change in state which would require accessing methods that send commands to peripheral microcontrollers and facilitate the failure and latency. The plants do not have latency in this example.

For the solenoid, we set its device to “RP2040”, which must perfectly match a device described in the device configuration file. We set the “pin” to 8 as a convenience. It will become an attribute of the `SolenoidDriver` for use in building commands, but is not called anywhere in the background. If we have data that there is a 5% chance that the solenoid is stuck (i.e. cannot change states from its current state) then we can reflect this in the `stuck_probability` field. The `cta_latency` field reflects the delay between a sent command and an actual change in the hardware state. There are currently two options for this field, either a normal distribution or a uniform distribution, with **values in milliseconds**. Here we will use a normal distribution with a mean of 10 milliseconds and a standard deviation of 3 milliseconds. Sample fields for the latency are given below:

```

cta_latency = {distribution='normal', mean=a, std=b}

cta_latency = {distribution='uniform', min=a, max=b}

```

The last nested dictionary in the actuator configuration file describes the effects of the actuator on the system variables. In the associated driver, we will write a method for each of these variables, describing the effect of the actuator on the system dynamics based on the state and time. The underlying code will search the “effect.types” field for the letter “p”, which means persistent.

`hilsim` takes the view that there are two effects that a system variable can experience from an actuator: persistent and transient. A persistent effect will be stored in the memory of the world state. For example, when the solenoid injects CO₂ or plants scrub CO₂, this permanently effects the CO₂ in the system. Transient effects, on the other hand, are solely time or state dependent. For example, current is never persistent. When the solenoid turns on, it draws current, when it is off, it does not draw current.

The only real character that matters in the field is “p”. If there is a “p” then the effect on that variable will be marked as persistent for the given actuator. If there is no “p” then the effects are assumed to be transient. I have also used “t” and “s” in the example to document the effects further dependency on time and state, but the are unused.

3.2.4 Actuator Drivers

```

from hilsim.core.actuators import ActuatorBase
from hilsim.core.world_state import WorldState
from hilsim.core.controls import Controls

class Plants(ActuatorBase):

    def build_command(self, target_state: str) -> str:
        """Create command to be sent to the microcontroller"""
        # Add code here to create a package (str) that a microcontroller
        # can interpret and return it:
        return None

    def effect_CO2(self) -> float:
        """sample effect method 1"""
        if self.hardware_state == "alive":

```

```

        return self.dt - 5

#- - - - -

from hilsim.core.actuators import ActuatorBase

class CO2Solenoid(ActuatorBase):
    """Solenoid driver to create commands and update world state"""

    def build_command(self, target_state: str) -> str:
        """Create command to be sent to the microcontroller"""
        pin = str(self.settings["pin"])
        if target_state == "open":
            package = pin + "," + str(1)
        else:
            package = pin + "," + str(0)
        return package

    def effect_CO2(self) -> float:
        """Define the physical effects of the CO2 solenoid on CO2"""
        if self.hardware_state == "open":
            return 1000 self.dt
        else:
            return 0

    def effect_current(self) -> float:
        """Define the physical effects of the CO2 solenoid on system current"""
        if self.hardware_state == "open":
            return 1001.6
        else:
            return 0

```

The actuator drivers have four important attributes. To understand them, it is necessary to elaborate on the system dynamics. At runtime, a thread splits off to run the background dynamics. It updates very quickly (set to 5 milliseconds, but this slips due to the nature of non-RTOS environments) by calling the effect methods of each actuator, allocating the effect into the system memory or as a temporary contribution, and then updating the system state. The ActuatorBase class contains an attribute `self.dt` which describes the time between the last call to the effect method and the current call.

1. `self.dt`: (seconds) This is the change in time since the last call to the effect methods and the current call. It is best used for rates. If CO₂ increases by 1000 ppm a second, then the return value for the effect method of an open CO₂ solenoid should be 1000 `self.dt`.
2. `self.cst`: (seconds) This is the time elapsed over the current hardware state. if a solenoid has been open for 5 seconds, then `self.cst` is 5 seconds.
3. `self.hardware_state`: (string) this is the current, actual state of the device and is used as the truth value for modeling system dynamics.
4. `self.world_state`: Provides access to the world state of the system, allowing the developer to call the `get(<variable_name>)` method to obtain other system variables. There are locks on this method, so it is safe to call even with the background dynamics thread is running.

For each actuator, it is necessary to write a `build_command(<target_state>)` method. This takes in the target state and constructs the actual package to be sent over serial for the microcontroller to interpret. This method is included so the user can send custom packages. While this method only builds the package, it is called by the `send_command(<target_state>)` method included in the base class, which takes in a target state, builds the custom package, and sends the command to the correct microcontroller.

3.2.5 Device Configuration File

The last hardware we have to describe are the peripheral microcontrollers. The most important field for a microcontroller is its `device_id`, or rather its device file. This will typically populate as something like `/dev/ttyACM0`, but it is recommended to use the actual device id rather than ACM with the COM port number because they can change. As stated earlier, the basic sim mode is not currently supported, which would target the serial port described by the `fake_device_id`. To simulate a device, use the `super_sim` mode and its associated failure rate, which describes how often the communication between the host and peripheral microcontroller fails.

Most of the settings are standard serial port configurations such as the baud rate and timeout. The `read_time` is quantifies how long the host will wait for a valid confirmation for the message it sent (in seconds). There are two other fields: `hb_send_msg` and `hb_receive_msg` that describe the message schema for a heart beat. This allows the developer to circumvent the standardized identical message schema used for confirmation of custom packages and use common heart beat idioms like “ping” and “pong.” These messages are used when the `heart_beat()` method is called on a device.

The `advanced_communication` field defines the automatic recovery method for packages that fail to send or receive a valid confirmation. There can be n retries, followed by m software cycles, and k power cycles. Currently, the need for these cycles will show up in method logs but is not actually supported to keep the software hardware agnostic. A future release will allow for rapid integration of software cycling and power cycling of peripheral microcontrollers. With this in mind, we arrive at the following device configuration file:

```
enabled_devices = ["RP2040"]

[advanced_communication]
retries          = 3
connection_cycles = 1
power_cycles     = 1

[devices.RP2040]
sim              = false
super_sim        = true
super_sim_fr     = 0.0
baud_rate        = 115200
device_ID        = "/dev/ttyACM0"
fake_device_ID   = ""
time_out         = 1.0
read_time        = 1.0
hb_send_msg      = "ping"
hb_receive_msg   = "pong"
```

3.2.6 World State Configuration File

The simplest of the configuration files, the world state configuration file holds of all the system variables, their initial values for simulation, and their units. Note that it is necessary to include all of the variables that the sensors read, even if we are not interested in their data.

```
[variables]
CO2              = {initial = 210, units = "ppm"}
relative_humidity = {initial = 60, units = "percent"}
temperature       = {initial = 24, units = "C"}
current           = {initial = 0, units = "mA"}
voltage           = {initial = 12, units = "V"}
power             = {initial = 0, units = "mW"}
```

3.2.7 Main Function

The main function contains the main logic of the system. It is automatically imported and ran when the cli tool `hilsim run` is ran from the project root. It has access to the sensor manager, which has `read_all()` and `read(<sensor_name>)` methods to interact with sensors. The `read_all()` method will

read all of the enabled sensors. Both read methods return a tuple containing the data (in the form of a nested dictionary) and the time of the reading(s). To track the sensor data (flag errors, perform recovery when supported on a future release) pass the data from `read_all()` into the sensors tracker's `track(<data>)` method.

Logging happens automatically. Logs will be placed in a folder called `logs` in the project root. The name of every log file contains the date and time it was created. To create a csv of sensor data, call the data manager's `add_data_to_file(<data>, <time>)` method. Data files are placed in a folder named `data` in the project root. To send a command to an actuator, use the `send_command(<target_state>)` on the actuator instance, which is contained in the `actuators` dictionary where the key is a string of the actuator's name, and the value is an instance of the actuator. Here is an example main function for our injection system.

```

"""main function"""

import time

def main(ctx):

    # Break up context
    sensor_manager = ctx["sensor_manager"]
    sensor_tracker = ctx["sensor_tracker"]
    actuators      = ctx["actuators"]
    world_state    = ctx["world_state"]
    data_manager   = ctx["data_manager"]
    log_manager    = ctx["log_manager"]
    rich_table     = ctx["table"]
    console        = ctx["console"]

    # Get instance of CO2 solenoid actuator
    co2_solenoid = actuators["Solenoid"]

    # create flag to terminate test
    test_complete = False
    start_terminal_clock = False
    open_command_sent = False
    close_command_sent = False

    while not test_complete:

        # Read sensor suite, add data to file, output to rich table, and track errors
        data, time_val = sensor_manager.read_all()
        data_manager.add_data_to_file(data, time_val)
        rich_table.add_row(*data_manager.convert_data_to_str_iterable(data, time_val))
        console.print(rich_table)
        sensor_tracker.track(data)

        # Pull CO2 data:
        CO2_data = data["SCD30"]["CO2"].value

        # Open solenoid when CO2 dips below 200 ppm, careful that
        # Readings can return none values
        if CO2_data is not None:
            if CO2_data < 200 and not open_command_sent:
                co2_solenoid.send_command("open")
                open_command_sent = True

        # Close solenoid when CO2 rises above 1200 ppm,
        # and terminate test 2 seconds later
        if CO2_data > 1200 and not close_command_sent:
            co2_solenoid.send_command("close")

```

```

close_command_sent = True
start_terminal_clock = True
start_time = time.time()

# Terminate 2 seconds after CO2 becomes greater than 1200 ppm
if start_terminal_clock:
    if (time.time() - start_time) > 2:
        test_complete = True

# Sleep 0.2 seconds between sensor reads and logic evaluation
time.sleep(0.2)

```

With the main function built, we simply run `hilsim run` from the project root to run the simulation.

3.2.8 Results

Graphing the data generated by the simulation:

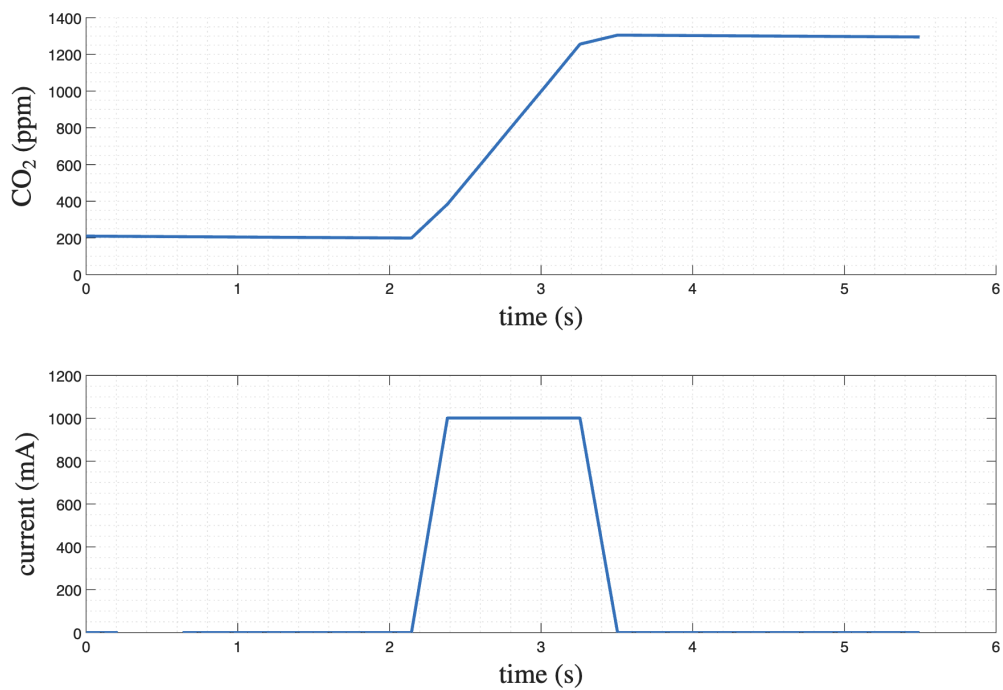


Figure 3: Results of CO₂ injection simulation